# Kaarigar Setu

Java implementation document

The Java parts of the project: the object-oriented design, exceptions, collections, file handling, JDBC, servlets and JSP, the JavaFX console and the cryptography and security work written in Java. Every code listing is taken directly from the source files at the time of writing, with its file and line number.

| Measure | Value |
|---|---|
| Web application (kaarigarsetu-java) | 121 main source files, 8,357 lines of Java; Jakarta Servlet 6 on Tomcat 10.1; compiled for Java 17 (built with JDK 21) |
| Test code | 49 test files, 5,488 lines: 322 JUnit 5 tests (unit, PostgreSQL integration, fuzz, architecture guards, simulation) |
| JavaFX moderator console (kaarigarsetu-moderator-console) | 10 files, 1,008 lines; JavaFX 21.0.5; 13 tests |
| Language constructs in main code | 39 files declare records, 6 enums, 9 interfaces, 3 abstract classes, 155 try-with-resources blocks, 92 prepared-statement calls, 62 Optional uses, 16 text blocks |
| Libraries (runtime) | Jakarta Servlet and JSTL, PostgreSQL JDBC 42.7, HikariCP 5.1, Gson 2.11, ZXing 3.5 (QR codes), SLF4J; no framework, no ORM |
| Build and quality | Maven; JaCoCo coverage; SpotBugs with find-sec-bugs and CycloneDX through the sast profile; Cargo for running Tomcat in development |

## What is highlighted

- **The five topics of the Review 1 slides, as built:** abstract class with polymorphic scoring, user-defined exceptions, the Collections Framework, file handling with java.nio, JDBC with PreparedStatement, servlets, JSP, and a JavaFX console.
- **What grew beyond the slides:** a layered design with dependency injection through constructors, a connection pool with thread-bound transactions, deny-by-default filters, 39 immutable record types, and a full test suite that includes reflection-driven SQL-injection fuzzing.
- **The Java written for the information-security review:** RFC 8785 canonical JSON, an RSA key ring with rotation, a hash-chained audit file with database anchors, HMAC live codes, a security monitor, moderator vetting of employers, and an attack simulation that calls the real scoring code.

## How to read the listings

Each listing has a short explanation above it and, below it, its source file, the line it starts on, and the Java features it demonstrates. The last section is an index of listings by feature, and the section before it lists where the finished code differs from the sketches on the Review 1 slides.

# Contents


| Section | Topic |
|---|---|
| 1 | Architecture and package structure |
| 2 | Object-oriented design: abstract class, inheritance, polymorphism |
| 3 | User-defined exceptions and one place to handle them |
| 4 | Collections, generics, records and modern Java |
| 5 | File handling with java.nio |
| 6 | JDBC: prepared statements, pooling and transactions |
| 7 | Servlets and filters |
| 8 | JSP and JSTL |
| 9 | The JavaFX moderator console |
| 10 | Cryptography and key handling with the Java platform API |
| 11 | Concurrency and scheduled work |
| 12 | Java built for the security review |
| 13 | Testing the Java code |
| 14 | Where the code differs from the Review 1 slides |
| 15 | Index of listings by Java feature |

# 1. Architecture and package structure

The application is a single WAR with five layers. There is no framework: services receive their collaborators through constructors, which is also what makes them testable with stand-ins.

| Layer | Responsibility | Main classes (package) |
|---|---|---|
| Presentation | JSP pages with JSTL; a JavaFX desktop console | WEB-INF/jsp/*.jsp; ModeratorConsoleApp (console module) |
| Request handling | Filters (headers, CSRF, access control) and servlets | SecurityHeadersFilter, CsrfFilter, AuthFilter (security/web); AuthServlet, WorkerServlet, AssessmentServlet, AttestationServlet, CredentialServlet, ModeratorServlet |
| Services | Business rules | AssessmentService and six trade subclasses, AttestationWorkflowService, CredentialService, TrustScoreService, ModeratorService, AuthService |
| Data access | JDBC, one DAO per table, a pooled connection provider with transactions | WorkerDao, AttestationDao, CredentialDao, AuditLogDao (dao); PooledConnectionProvider (config) |
| Cross-cutting | Security primitives and helpers | SignatureService, PasswordHasher, PhoneProtector, RateLimiter, AuditLogger, ChainedAuditFileSink (security); CanonicalJson (util) |

## Package map

| Package (com.kaarigarsetu.*) | Files | Purpose |
|---|---|---|
| assessment | 15 | abstract AssessmentService, six trade subclasses, scoring dispatcher, skill bands, the assessment pipeline, AssessmentServlet |
| security | 20 | OTP, login, password hashing, phone encryption, signatures, rate limiting, audit logging, AuthFilter, live-code tokens |
| dao | 10 | JDBC data access (one class per table) built on JdbcDaoSupport |
| model | 13 | immutable data carriers: every one is a Java record |
| exception | 13 | the user-defined exceptions |
| web | 10 | BaseServlet, page servlets, CSRF and header filters, application wiring (AppContext), security-event recording |
| trust | 7 | ReputationScoring (the trust formula), TrustScoreService, TrustGraphService (adjacency list), Edge |
| attestation | 7 | the employer confirmation workflow, the in-memory store, AttestationServlet |
| config / util | 5 / 5 | settings, pooled connections and transactions / canonical JSON, audit writer, credential exporter, WAV validation |
| credential, moderator, worker, employer, ml, privacy, admin | 3, 3, 2, 2, 2, 1, 2 | signed credentials, moderator review and the security monitor, registration and passport, the Python ML-service client, data erasure, demo seeding and the offline audit verifier |

## Request flow in Java terms

A request enters Tomcat and passes three filters in the order declared in web.xml (Listing 1): security headers, then the CSRF check, then access control. Only then does a servlet run. Every servlet extends BaseServlet, which turns any exception into a short coded response and offers helpers for JSON, path segments and throttling. Servlets call services; services call DAOs; DAOs use the pooled connection provider.

### Listing 1. Filter order is declared in web.xml

Filters run in the order their mappings appear. Putting the security-header filter first means even a rejected request carries the headers.

```
<filter>
    <filter-name>SecurityHeadersFilter</filter-name>
    <filter-class>com.kaarigarsetu.web.SecurityHeadersFilter</filter-class>
</filter>
<filter>
    <filter-name>CsrfFilter</filter-name>
    <filter-class>com.kaarigarsetu.web.CsrfFilter</filter-class>
</filter>
<filter>
    <filter-name>AuthFilter</filter-name>
    <filter-class>com.kaarigarsetu.security.AuthFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>SecurityHeadersFilter</filter-name>
```

**Source:** web: WEB-INF/web.xml, line 9. **Java features:** Servlet Filter API, declarative configuration.

# 2. Object-oriented design: abstract class, inheritance, polymorphism

The scoring engine is the project's main object-oriented structure, exactly as the Review 1 slides described it, but with the shared rule made final so that no subclass can change how a score is computed.

### Listing 2. AssessmentService: one shared rule, two things each trade must supply

supportedTrade() and feedbackFor() are abstract; assess() and summarize() are public final. This is the template-method pattern: the algorithm (exactly 10 answers, scores kept between 0 and 100, the mean is the result) cannot be overridden, only the wording and the trade name vary.

```
12 public abstract class AssessmentService {
13     /** Every assessment is exactly this many questions (the rubric files carry the same number). One
         definition, used by scoring, completion and credential rules. */
14     public static final int QUESTIONS_PER_ASSESSMENT = 10;
15
16     public abstract String supportedTrade();
17
18     public final AssessmentResult assess(List<String> answers) {
19         Objects.requireNonNull(answers, "answers must not be null");
20         if (answers.size() != QUESTIONS_PER_ASSESSMENT) {
21             throw new IllegalArgumentException("An assessment must contain exactly " + QUESTIONS_PER_ASSESSMENT
       + " answers");
22         }
23
24         long answered = answers.stream().filter(this::hasContent).count();
25         BigDecimal score = BigDecimal.valueOf(answered * 100.0 / answers.size())
26                 .setScale(2, RoundingMode.HALF_UP);
27         return new AssessmentResult(supportedTrade(), score, feedbackFor(score));
28     }
29
30     /**
31      * Template step for the ML-scored pipeline: the five per-question scores (0-100, from the scoring service)
         are
32      * averaged by this one shared rule, and each trade subclass supplies its own feedback wording.
33      */
34     public final AssessmentResult summarize(List<BigDecimal> perQuestionScores) {
35         Objects.requireNonNull(perQuestionScores, "scores must not be null");
36         if (perQuestionScores.size() != QUESTIONS_PER_ASSESSMENT) {
37             throw new IllegalArgumentException("An assessment must contain exactly " + QUESTIONS_PER_ASSESSMENT
       + " scores");
38         }
39         BigDecimal total = BigDecimal.ZERO;
40         for (BigDecimal score : perQuestionScores) {
41             if (score == null || score.signum() < 0 || score.compareTo(BigDecimal.valueOf(100)) > 0) {
42                 throw new IllegalArgumentException("each score must be between 0 and 100");
43             }
44             total = total.add(score);
45         }
46         BigDecimal mean = total.divide(BigDecimal.valueOf(perQuestionScores.size()), 2, RoundingMode.HALF_UP);
47         return new AssessmentResult(supportedTrade(), mean, feedbackFor(mean));
48     }
49
50     protected abstract String feedbackFor(BigDecimal score);
```

**Source:** assessment/AssessmentService.java, line 12. **Java features:** abstract class, final methods, template method, BigDecimal arithmetic, defensive validation.

### Listing 3. A trade subclass: small and only about wording

There are six of these (Electrician, Plumber, Carpenter, Mason, Welder, Painter). Adding a trade is one rubric file, one line in trades.json and a class like this.

```
 5 public class PlumberAssessmentService extends AssessmentService {
 6     @Override
 7     public String supportedTrade() {
 8         return "Plumber";
 9     }
10
11     @Override
12     protected String feedbackFor(BigDecimal score) {
13         return score.compareTo(new BigDecimal("80")) >= 0
14                 ? "Strong plumbing diagnostics and installation responses."
15                 : "Review leak diagnosis, fittings, and safe installation practice.";
16     }
17 }
```

**Source:** assessment/PlumberAssessmentService.java, line 5. **Java features:** inheritance, @Override, protected hook method.

**Listing 4. Polymorphic dispatch through a common reference**

The dispatcher holds a List of AssessmentService and picks the subclass for the worker's trade with a stream; the call through the base type runs the right override at run time.

```
15 public AssessmentResult score(String trade, List<String> answers) {
16     return resolve(trade).assess(answers);
17 }
18
19 /** Polymorphic: the trade's own subclass averages the ML scores and writes its own feedback. */
20 public AssessmentResult summarize(String trade, List<java.math.BigDecimal> perQuestionScores) {
21     return resolve(trade).summarize(perQuestionScores);
22 }
23
24 private AssessmentService resolve(String trade) {
25     String normalizedTrade = normalize(trade);
26     return services.stream()
27             .filter(service -> normalize(service.supportedTrade()).equals(normalizedTrade))
28             .findFirst()
29             .orElseThrow(() -> new IllegalArgumentException("Unsupported trade: " + trade));
30 }
31
32 private String normalize(String value) {
33     if (value == null || value.isBlank()) {
34         throw new IllegalArgumentException("trade must not be blank");
35     }
36     return value.trim().toLowerCase(Locale.ROOT);
```

**Source:** assessment/AssessmentScoringDispatcher.java, line 15. **Java features:** polymorphism, streams, Optional.orElseThrow, constructor-injected List.

## Interfaces used for the same purpose

Interfaces keep services independent of the technology behind them: OtpSender (console sender and HTTP SMS sender), OtpChallengeStore (in-memory and database), AuditFileSink (plain and hash-chained writers), AuditLogRepository, ConnectionProvider, MlGateway (the Python ML service) and, in the console module, ModeratorApi. Each has an implementation for production and a stand-in for tests, which is why the 322 tests can run without a network.

# 3. User-defined exceptions and one place to handle them

Thirteen exceptions model domain events (a wrong code, a duplicate attestation, a low trust score, a missing consent) rather than technical failures. All extend RuntimeException; a single method turns each into the right HTTP status.

### Listing 5. InvalidOTPException

The smallest example: a named domain event with one constructor. The other twelve have the same shape.

```
 3 /** Raised when an attestation OTP cannot be accepted. */
 4 public class InvalidOTPException extends RuntimeException {
 5     public InvalidOTPException(String message) {
 6         super(message);
 7     }
 8 }
```

**Source:** exception/InvalidOTPException.java, line 3. **Java features:** user-defined exception, unchecked exception.

### Listing 6. Throwing the exception from a switch expression

The OTP service decides the result of a code check with a switch over an enum and throws the matching exception.

```
95     case INVALID -> throw new InvalidOTPException("OTP is invalid");
96     case LOCKED -> throw new InvalidOTPException("OTP is locked");
97     case UNAVAILABLE -> throw new InvalidOTPException("OTP is unavailable, expired, or locked");
98 }
```

**Source:** security/OtpService.java, line 95. **Java features:** switch expression with arrows, enum, throw expression.

### Listing 7. DuplicateAttestationException and a synchronised check

The store refuses the same employer confirming the same worker twice; the check and the insert happen inside one synchronized method so two simultaneous requests cannot both pass.

```
23 public synchronized void add(Attestation attestation) {
24     Objects.requireNonNull(attestation, "attestation must not be null");
25     boolean duplicate = attestations.stream().anyMatch(existing ->
26             existing.workerId().equals(attestation.workerId())
27                     && existing.employerId().equals(attestation.employerId()));
28     if (duplicate) {
29         throw new DuplicateAttestationException("An attestation already exists for this worker and employer");
30     }
31     attestations.add(repository.save(attestation));
32 }
```

**Source:** attestation/AttestationStore.java, line 23. **Java features:** synchronized, streams (anyMatch), user-defined exception.

### Listing 8. ApiErrors: every exception becomes a status and a short code

One method maps each exception type to an HTTP status and a stable error code. An exception nobody expected becomes a 500 with a random reference id logged server-side, so no stack trace or internal detail ever reaches the client.

```
public static void write(HttpServletResponse response, Throwable failure) throws IOException {
    if (response.isCommitted()) return;
    Map<String, Object> body = new LinkedHashMap<>();
    int status;
    String code;
    if (failure instanceof ConsentRequiredException) { status = 403; code = "consent_required"; }
    else if (failure instanceof InvalidOTPException) { status = 401; code = "invalid_otp"; }
    else if (failure instanceof AuthenticationFailedException) { status = 401; code = "invalid_credentials"; }
    else if (failure instanceof ForbiddenException) { status = 403; code = "forbidden"; }
    else if (failure instanceof LowTrustScoreException) { status = 403; code = "low_trust"; }
    else if (failure instanceof RateLimitExceededException limited) {
        status = 429; code = "rate_limited";
        response.setHeader("Retry-After", String.valueOf(limited.retryAfterSeconds()));
    }
    else if (failure instanceof UnintelligibleAudioException) { status = 422; code = "unintelligible"; }
    else if (failure instanceof ValidationException) { status = 400; code = "invalid_request"; }
    else if (failure instanceof NotFoundException) { status = 404; code = "not_found"; }
    else if (failure instanceof DuplicateAttestationException) { status = 409; code = "duplicate"; }
    else if (failure instanceof ConflictException) { status = 409; code = "conflict"; }
    else if (failure instanceof ExternalServiceException) { status = 503; code = "upstream_unavailable"; }
    else {
        byte[] ref = new byte[4];
        RANDOM.nextBytes(ref);
        String id = HexFormat.of().formatHex(ref);
        LOG.log(System.Logger.Level.ERROR, "Unhandled error ref=" + id, failure);
        status = 500; code = "internal";
        body.put("ref", id);
    }
    body.put("error", code);
    send(response, status, body);
```

**Source:** web/ApiErrors.java, line 35. **Java features:** pattern matching for instanceof (Java 16), System.Logger, HexFormat, SecureRandom.

# 4. Collections, generics, records and modern Java

The Collections Framework appears wherever order, uniqueness or fast lookup matters.

### Listing 9. The trust graph as an adjacency list

A Map from node to the List of its outgoing edges, filled with computeIfAbsent. The accessor returns an immutable copy so callers cannot change the graph.

```
12 public class TrustGraphService {
13     private final Map<String, List<Edge>> adjacencyList = new HashMap<>();
14     private final TrustGraphRepository repository;
15
16     public TrustGraphService() {
17         this(edge -> { });
18     }
19
20     public TrustGraphService(TrustGraphRepository repository) {
21         this.repository = java.util.Objects.requireNonNull(repository, "repository must not be null");
22     }
23
24     public void addEdge(Edge edge) {
25         repository.save(edge);
26         adjacencyList.computeIfAbsent(edge.source(), ignored -> new ArrayList<>()).add(edge);
27         adjacencyList.computeIfAbsent(edge.target(), ignored -> new ArrayList<>());
28     }
29
30     public Map<String, List<Edge>> adjacencyList() {
31         Map<String, List<Edge>> copy = new HashMap<>();
32         adjacencyList.forEach((node, edges) -> copy.put(node, List.copyOf(edges)));
33         return Map.copyOf(copy);
```

**Source:** trust/TrustGraphService.java, line 12. **Java features:** Map>, HashMap, computeIfAbsent, List.copyOf / Map.copyOf, record Edge.

### Listing 10. Sliding-window rate limiter

Each key owns a Deque of hit times inside a ConcurrentHashMap; compute() makes the trim-and-count step atomic per key without a global lock.

```
21 public void check(String key, int max, Duration window) {
22     long now = clock.millis();
23     long windowMs = window.toMillis();
24     int[] retryAfter = {0};
25     hits.compute(key, (k, existing) -> {
26         Deque<Long> queue = existing == null ? new ArrayDeque<>() : existing;
27         while (!queue.isEmpty() && now - queue.peekFirst() >= windowMs) queue.pollFirst();
28         if (queue.size() >= max) {
29             retryAfter[0] = (int) Math.max(1, (windowMs - (now - queue.peekFirst()) + 999) / 1000);
30             return queue;
31         }
32         queue.addLast(now);
33         return queue;
34     });
35     if (retryAfter[0] > 0) throw new RateLimitExceededException("Too many requests", retryAfter[0]);
36 }
37
38 public void evictExpired(Duration maxWindow) {
39     long now = clock.millis();
40     hits.entrySet().removeIf(e -> e.getValue().isEmpty() || now - e.getValue().peekLast() >=
          maxWindow.toMillis());
```

**Source:** security/RateLimiter.java, line 21. **Java features:** ConcurrentHashMap.compute, ArrayDeque, lambdas, custom exception carrying retry-after.

### Listing 11. Canonical JSON: a TreeMap sorts members

RFC 8785 sorts object members by their UTF-16 code units; Java's String ordering is exactly that, so a TreeMap gives the canonical order directly.

```
54          JsonObject object = element.getAsJsonObject();
55          Map<String, JsonElement> sorted = new TreeMap<>();              // String order is UTF-16 code unit
        order, as the RFC requires
56          object.entrySet().forEach(entry -> sorted.put(entry.getKey(), entry.getValue()));
57          out.append('{');
58          boolean first = true;
59          for (Map.Entry<String, JsonElement> entry : sorted.entrySet()) {
60              if (!first) out.append(',');
61              first = false;
62              string(entry.getKey(), out);
63              out.append(':');
64              write(entry.getValue(), out);
65          }
66          out.append('}');
67      }
68  }
```

**Source:** util/CanonicalJson.java, line 54. **Java features:** TreeMap, Map.Entry, recursion over JsonElement.

### Listing 12. Records for immutable data

Parameters, feedback, employer standing and the report are records: immutable, with equals, hashCode and toString generated. 39 source files declare records, including all 13 model classes.

```
36  public record Params(double priorMean, double priorWeight, double lowPrior, int fullConfidenceEmployers, double
        recencyDays, double recencyFloor,
37                       double leniencyStrength, double leniencyCap, int minRatedForCalibration, double
        integrityPenalty, double integrityPenaltyCap,
38                       double unratedSatisfaction, double unratedWeightFactor, boolean
        excellentNeedsVettedEmployer) {
39      public static Params defaults() {
40          return new Params(0.60, 1.5, 0.25, 3, 365.0, 0.15, 0.5, 0.10, 3, 5.0, 15.0, 0.55, 0.5, true);
41      }
42  }
43
44  /** Just under the Excellent band (80). */
45  static final double HELD_BELOW_EXCELLENT = 79.99;
46
47  /** One approved attestation about the worker. Ratings are 1..5 or null; wouldHireAgain may be null. */
48  public record Feedback(long attestationId, long workerId, long employerId, Integer quality, Integer punctuality,
         Integer conduct,
49                         Boolean wouldHireAgain, Integer durationDays, Instant at, double anomaly) {}
```

**Source:** trust/ReputationScoring.java, line 36. **Java features:** records with static factory, compact immutable carriers, boxed Integer / Boolean for 'absent'.

### Listing 13. A text block for SQL

Multi-line SQL is written as a text block so it stays readable, and every value still goes through a ? parameter.

```
public java.util.List<EmployerSummary> listForVetting(boolean onlyUnvetted, int limit) {
    String sql = """
            SELECT e.employer_id, e.name, e.verified, e.verified_by, e.created_at,
                   COUNT(a.attestation_id) FILTER (WHERE a.status = 'APPROVED') AS approved
            FROM employers e LEFT JOIN attestations a ON a.employer_id = e.employer_id
            WHERE e.erased_at IS NULL AND (? = FALSE OR e.verified = FALSE)
            GROUP BY e.employer_id
            ORDER BY approved DESC, e.created_at ASC, e.employer_id ASC
            LIMIT ?
            """;
    try (var connection = connectionProvider.getConnection();
         PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setBoolean(1, onlyUnvetted);
        statement.setInt(2, limit);
        try (ResultSet resultSet = statement.executeQuery()) {
            java.util.List<EmployerSummary> rows = new java.util.ArrayList<>();
            while (resultSet.next()) {
                rows.add(new EmployerSummary(resultSet.getLong("employer_id"), resultSet.getString("name"),
      resultSet.getBoolean("verified"),
                        resultSet.getString("verified_by"), resultSet.getInt("approved"),
      resultSet.getTimestamp("created_at").toLocalDateTime()));
            }
            return rows;
        }
    } catch (SQLException exception) {
        throw dataAccessException("list employers for vetting", exception);
    }
}
```

**Source:** dao/EmployerDao.java, line 133. **Java features:** text blocks, try-with-resources, PreparedStatement, record result type.

## Other modern constructs used throughout

| Construct | Where |
|---|---|
| Streams and Optional | DAOs return Optional; dispatcher, alert rules and verification use streams |
| java.time with an injected Clock | services take a Clock, so time-dependent rules (expiry, 5-minute live codes) are tested with a controllable clock |
| var, generics, functional interfaces | inTransaction(Supplier); @FunctionalInterface AuditLogRepository and AuditFileSink, used as lambdas in tests |
| java.net.http.HttpClient | ML-service client, SMS gateway client and the console's REST client |
| Reflection | the SQL-injection fuzz test calls every public DAO method that takes a String (Section 13) |

# 5. File handling with java.nio

Three file jobs: an audit trail that cannot be edited quietly, the export of a signed credential, and an offline checker.

### Listing 14. The simple append-only audit writer

The first version from the review: Files.writeString with CREATE and APPEND, one timestamped line per event.

```
29      public synchronized void append(String event) throws IOException {
30          if (event == null || event.isBlank()) {
31              throw new IllegalArgumentException("event must not be blank");
32          }
33          Path parent = logFile.toAbsolutePath().getParent();
34          if (parent != null) {
35              Files.createDirectories(parent);
36          }
37          Files.writeString(logFile, Instant.now(clock) + " " + event.strip() + System.lineSeparator(),
38                  StandardCharsets.UTF_8, StandardOpenOption.CREATE, StandardOpenOption.APPEND);
39      }
40  }
```

**Source:** util/AuditLogWriter.java, line 29. **Java features:** java.nio.file.Files, StandardOpenOption, Clock, synchronized.

### Listing 15. The hash-chained audit file

Each line holds the SHA-256 of the previous line, so editing, deleting or reordering any line breaks the chain. A FileChannel with an exclusive FileLock and force(true) makes every append atomic and durable, and a keyed head file stores an HMAC of the newest line.

```
52  public void append(String event) throws IOException {
53      lock.lock();
54      try {
55          Files.createDirectories(java.util.Objects.requireNonNull(file.getParent(), "audit file needs a parent
          directory"));
56          try (FileChannel channel = FileChannel.open(file, StandardOpenOption.CREATE, StandardOpenOption.WRITE,
          StandardOpenOption.APPEND);
57                 FileLock ignored = channel.lock()) {
58              List<String> existing = Files.readAllLines(file, StandardCharsets.UTF_8);
59              long seq = existing.size() + 1;
60              JsonObject line = new JsonObject();
61              line.addProperty("seq", seq);
62              line.addProperty("ts", clock.instant().toString());
63              line.addProperty("event", event);
64              line.addProperty("prev", existing.isEmpty() ? GENESIS : sha256(existing.get(existing.size() - 1)));
65              String text = Json.toJson(line);
66              channel.write(ByteBuffer.wrap((text + "\n").getBytes(StandardCharsets.UTF_8)));
67              channel.force(true);
68              Files.writeString(headFile, headMac(seq, sha256(text)), StandardCharsets.UTF_8);
69          }
70      } finally {
71          lock.unlock();
72      }
73  }
```

**Source:** security/ChainedAuditFileSink.java, line 52. **Java features:** FileChannel, FileLock, StandardOpenOption.APPEND, ReentrantLock, try-with-resources, JSON via Gson.

### Listing 16. Anchors: catching a rollback

A copy of an older log together with its older head record is internally consistent, so the chain alone cannot catch it. The newest line's hash is also pinned in the database; verification compares the file against those anchors.

```
131     for (Anchor anchor :
          anchors.stream().sorted(java.util.Comparator.comparingLong(Anchor::seq).reversed()).toList()) {
132         if (anchor.seq() < 1) continue;
133         if (anchor.seq() > lines.size()) {
134             return VerifyResult.bad(lines.size(), lines.size(), "file has " + lines.size() + " lines but the
          database anchored line " + anchor.seq() + ": rolled back or truncated");
135         }
136         if (!MessageDigest.isEqual(hashes.get((int) anchor.seq() - 1).getBytes(StandardCharsets.US_ASCII),
          anchor.lineHash().getBytes(StandardCharsets.US_ASCII))) {
137             return VerifyResult.bad(lines.size(), anchor.seq(), "line " + anchor.seq() + " differs from the copy
           anchored in the database: file rewritten");
138         }
139     }
140     return VerifyResult.ok(lines.size());
141 } finally {
```

**Source:** security/ChainedAuditFileSink.java, line 131. **Java features:** MessageDigest.isEqual (constant-time), streams, Comparator, records.

### Listing 17. Exporting the signed credential

The signed document is written as UTF-8 JSON; a one-page PDF export also exists.

```
11 public Path exportJson(Path destination, String signedJson) throws IOException {
12     Objects.requireNonNull(destination, "destination must not be null");
13     Objects.requireNonNull(signedJson, "signedJson must not be null");
14     createParentDirectories(destination);
15     return Files.writeString(destination, signedJson, StandardCharsets.UTF_8);
16 }
```

**Source:** util/CredentialExporter.java, line 11. **Java features:** Files.writeString, Path, UTF-8 charsets.

### Listing 18. An offline auditor tool

AuditVerifyCli is a plain main method that checks a copy of the audit file, optionally against the database anchors. The secret key is read from the environment, never from the command line.

```
50 List<ChainedAuditFileSink.Anchor> anchors = useDb ? readAnchors() : List.of();
51 ChainedAuditFileSink.VerifyResult result = new ChainedAuditFileSink(file, key.getBytes(StandardCharsets.UTF_8),
       Clock.systemUTC()).verify(anchors);
52 if (result.valid()) {
53     System.out.println("INTACT lines=" + result.lines() + (useDb ? " anchors=" + anchors.size() : ""));
54 } else {
55     System.out.println("TAMPERED line=" + result.firstBadLine() + " of " + result.lines() + " reason=" +
         result.reason());
56     System.exit(1);
57 }
```

**Source:** admin/AuditVerifyCli.java, line 50. **Java features:** main method, exit codes, JDBC from a command-line tool.

# 6. JDBC: prepared statements, pooling and transactions

All persistence is plain JDBC. There is no ORM, so every statement is visible and every parameter is bound explicitly.

### Listing 19. JdbcDaoSupport: what every DAO shares

An abstract base class holding the connection provider and two helpers, so each DAO stays short.

```
12 abstract class JdbcDaoSupport {
13     protected final ConnectionProvider connectionProvider;
14
15     protected JdbcDaoSupport(ConnectionProvider connectionProvider) {
16         this.connectionProvider = connectionProvider;
17     }
18
19     protected DataAccessException dataAccessException(String action, SQLException exception) {
20         return new DataAccessException("Failed to " + action, exception);
21     }
22
23     protected Optional<LocalDateTime> optionalDateTime(ResultSet resultSet, String column) throws SQLException {
24         Timestamp timestamp = resultSet.getTimestamp(column);
25         return timestamp == null ? Optional.empty() : Optional.of(timestamp.toLocalDateTime());
26     }
27 }
```

**Source:** dao/JdbcDaoSupport.java, line 12. **Java features:** abstract class, Optional, java.sql types.

### Listing 20. A DAO method with an array parameter

AuditLogDao counts security events per type with event_type = ANY (?) and binds the list as a SQL array. Even a hostile type name is only a value.

```
64 public java.util.Map<String, Long> countByType(int minutes, java.util.Collection<String> eventTypes) {
65     String sql = "SELECT event_type, count(*) AS n FROM audit_log WHERE \"timestamp\" >= CURRENT_TIMESTAMP - (?
       * INTERVAL '1 minute') AND event_type = ANY (?) GROUP BY event_type";
66     try (var connection = connectionProvider.getConnection();
67          PreparedStatement statement = connection.prepareStatement(sql)) {
68         statement.setInt(1, minutes);
69         statement.setArray(2, connection.createArrayOf("varchar", eventTypes.toArray()));
70         try (ResultSet resultSet = statement.executeQuery()) {
71             java.util.Map<String, Long> counts = new java.util.LinkedHashMap<>();
72             while (resultSet.next()) counts.put(resultSet.getString("event_type"), resultSet.getLong("n"));
73             return counts;
74         }
75     } catch (SQLException exception) {
76         throw dataAccessException("count audit events", exception);
77     }
78 }
79
80 public java.util.Optional<Long> newestAgeMinutes(String eventType) {
81     String sql = "SELECT floor(EXTRACT(EPOCH FROM (CURRENT_TIMESTAMP - max(\"timestamp\"))) / 60) AS minutes
       FROM audit_log WHERE event_type = ?";
```

**Source:** dao/AuditLogDao.java, line 64. **Java features:** PreparedStatement, createArrayOf, try-with-resources, LinkedHashMap.

**Listing 21. Thread-bound transactions on a connection pool**

inTransaction opens one connection, stores it in a ThreadLocal so every DAO call on this thread joins it, commits on success and rolls back on any failure, adding a rollback problem as a suppressed exception instead of hiding the original.

```
43 public <T> T inTransaction(Supplier<T> work) {
44     if (current.get() != null) {
45         return work.get();   // join the surrounding transaction
46     }
47     try (Connection connection = dataSource.getConnection()) {
48         connection.setAutoCommit(false);
49         current.set(connection);
50         try {
51             T result = work.get();
52             connection.commit();
53             return result;
54         } catch (RuntimeException | Error failure) {
55             try {
56                 connection.rollback();
57             } catch (SQLException rollbackFailure) {
58                 failure.addSuppressed(rollbackFailure);
59             }
60             throw failure;
61         } finally {
62             current.remove();
63             connection.setAutoCommit(true);
64         }
65     } catch (SQLException exception) {
66         throw new DataAccessException("Failed to run transaction", exception);
67     }
68 }
```

**Source:** config/PooledConnectionProvider.java, line 43. **Java features:** HikariCP, ThreadLocal, generics with Supplier, commit / rollback, addSuppressed, finally.

The database also enforces rules the Java code relies on: the application connects as a restricted role (data changes only, no schema changes), and the audit table rejects UPDATE, DELETE and TRUNCATE with triggers. A test fails the build if any SQL string is assembled by concatenation (Section 13).

# 7. Servlets and filters

Jakarta Servlet 6 with annotations. Seven API servlets, one health servlet and six page servlets all extend BaseServlet; three filters guard every request.

### Listing 22. BaseServlet.service: the single error boundary

Overriding service() lets every servlet inherit one try/catch: container failures pass through, rate-limit refusals are recorded as security events, and everything else becomes a coded response.

```
26 protected void service(HttpServletRequest request, HttpServletResponse response) throws ServletException,
       IOException {
27     try {
28         super.service(request, response);
29     } catch (ServletException | IOException containerFailure) {
30         throw containerFailure;
31     } catch (Exception failure) {
32         if (failure instanceof com.kaarigarsetu.exception.RateLimitExceededException)
       SecurityEvents.record(request, "RATE_LIMITED");
33         ApiErrors.write(response, failure);
34     }
35 }
```

**Source:** web/BaseServlet.java, line 26. **Java features:** abstract class extends HttpServlet, template method, exception translation.

### Listing 23. A one-line throttle for public routes

Public routes call this to count requests per client address and window.

```
110 protected void throttle(HttpServletRequest request, String bucket, int max, Duration window) {
111     app().limiter.check(bucket + ":ip:" + app().phones.ipHash(clientIp(request)), max, window);
112 }
```

**Source:** web/BaseServlet.java, line 110. **Java features:** protected helper, java.time.Duration, hashed client address.

### Listing 24. AttestationServlet.doPost

A switch over the path decides the action; input is validated first (dates parsed, ratings required); the servlet itself holds no business logic.

```
22 protected void doPost(HttpServletRequest request, HttpServletResponse response) throws IOException {
23     long employerId = user(request).id();
24     JsonObject body = readJson(request);
25     switch (String.join("/", segments(request))) {
26         case "request-otp" -> {
27             LocalDate date;
28             try {
29                 date = LocalDate.parse(string(body, "workDate"));
30             } catch (DateTimeParseException malformed) {
31                 throw new ValidationException("Bad date");
32             }
33             String device = app().phones.deviceHash(String.valueOf(request.getHeader("User-Agent")),
       String.valueOf(request.getHeader("Accept-Language")));
34             Integer quality = optionalInt(body, "qualityRating"), punctuality = optionalInt(body,
       "punctualityRating"), conduct = optionalInt(body, "conductRating");
35             if (quality == null || punctuality == null || conduct == null) throw new ValidationException("Please
        rate quality, punctuality and conduct");
36             var requested = app().attestations.requestOtp(employerId, string(body, "workerPhone"), string(body,
       "jobRole"),
37                     optionalInt(body, "durationDays"), optionalBoolean(body, "wouldHireAgain"), date,
       clientIp(request), device,
38                     quality, punctuality, conduct);
39             json(response, 201, requested);
40         }
41         case "verify-otp" -> {
42             JsonElement id = body.get("attestationId");
43             if (id == null || !id.isJsonPrimitive() || !id.getAsJsonPrimitive().isNumber()) throw new
       ValidationException("attestationId is required");
44             json(response, 200, app().attestations.verifyOtp(employerId, id.getAsLong(), string(body, "code"),
       clientIp(request)));
45         }
46         default -> throw new NotFoundException("Not found");
47     }
48 }
```

**Source:** attestation/AttestationServlet.java, line 22. **Java features:** @WebServlet, switch with arrows, java.time.LocalDate, custom exceptions.

### Listing 25. AuthFilter: deny by default

A table of rules (method, path, allowed roles) is checked for every request. A route with no rule under /api needs a session; a signed-in user reaching another role's route is refused and recorded as ACCESS_DENIED.

```
81 public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) throws IOException,
       ServletException {
82     HttpServletRequest http = (HttpServletRequest) request;
83     HttpServletResponse out = (HttpServletResponse) response;
84     String path = http.getServletPath() + (http.getPathInfo() == null ? "" : http.getPathInfo());
85     boolean api = path.startsWith("/api/");
86     Optional<Rule> rule = RULES.stream().filter(r -> r.matches(http.getMethod(), path)).findFirst();
87
88     if (rule.isPresent() && rule.get().roles().isEmpty()) {
89         chain.doFilter(request, response);
90         return;
91     }
92     if (rule.isEmpty() && !api) {
93         chain.doFilter(request, response);   // unlisted non-API paths (error pages) are not data endpoints
94         return;
95     }
96
97     HttpSession session = http.getSession(false);
98     Optional<AuthenticatedUser> user = session == null ? Optional.empty() : Sessions.current(http);
99     if (user.isEmpty()) {
100        deny(http, out, api, 401, "unauthenticated");
101        return;
102    }
103    Instant loginAt = (Instant) session.getAttribute(Sessions.LOGIN_AT);
104    if (loginAt == null || Duration.between(loginAt, Instant.now()).compareTo(MAX_SESSION_AGE) > 0) {
105        session.invalidate();
106        deny(http, out, api, 401, "unauthenticated");
107        return;
108    }
109    if (rule.isPresent() && !rule.get().roles().contains(user.get().role())) {
110        com.kaarigarsetu.web.SecurityEvents.record(http, "ACCESS_DENIED");      // a signed-in user reaching for
       another role's endpoint
111        deny(http, out, api, 403, "forbidden");
112        return;
113    }
114    chain.doFilter(request, response);
115 }
116
117 private static void deny(HttpServletRequest request, HttpServletResponse response, boolean api, int status,
       String code) throws IOException {
118    if (api) ApiErrors.write(response, status, code);
```

**Source:** security/AuthFilter.java, line 81. **Java features:** jakarta.servlet.Filter, records, streams, Optional, Set.

### Listing 26. CsrfFilter

A POST, PUT, PATCH or DELETE must carry the session's token and a same-origin Origin header; otherwise 403 and a CSRF_REJECTED event.

```
25 public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) throws IOException,
       ServletException {
26     HttpServletRequest http = (HttpServletRequest) request;
27     if (SAFE.contains(http.getMethod())) {
28         chain.doFilter(request, response);
29         return;
30     }
31     HttpSession session = http.getSession(false);
32     if (session == null || !Sessions.csrfMatches(session, http.getHeader("X-CSRF-Token")) || !sameOrigin(http))
         {
33         SecurityEvents.record(http, "CSRF_REJECTED");
34         ApiErrors.write((HttpServletResponse) response, 403, "csrf");
35         return;
36     }
37     chain.doFilter(request, response);
38 }
39
40 private static boolean sameOrigin(HttpServletRequest request) {
```

**Source:** web/CsrfFilter.java, line 25. **Java features:** Filter, constant-time token comparison, URI parsing.

### Listing 27. SecurityEvents: audit without a flood

Each client address is recorded at most once per event type per minute by reusing the rate limiter as a gate, so an attacker cannot turn monitoring into a way to fill the log.

```
19 public static void record(HttpServletRequest request, String eventType) {
20     try {
21         AppContext app = AppContext.of(request.getServletContext());
22         String ipHash = app.phones.ipHash(clientIp(app, request));
23         try {
24             app.limiter.check("audit:" + eventType + ":" + ipHash, 1, ONE_PER);
25         } catch (RateLimitExceededException alreadyRecorded) {
26             return;
27         }
28         String actor = Sessions.current(request).map(u -> u.role().name().toLowerCase() + ":" +
       u.id()).orElse("anonymous");
29         String path = request.getServletPath() + (request.getPathInfo() == null ? "" : request.getPathInfo());
30         app.audit.log(eventType, actor, "method=" + request.getMethod() + " path=" + path + " client=" +
       ipHash.substring(0, 12));
31     } catch (RuntimeException monitoringFailure) {
32         System.getLogger("ks.security").log(System.Logger.Level.WARNING, "could not record " + eventType + ": "
       + monitoringFailure.getMessage());
33     }
34 }
35
36 /** The client address for rate limiting and the audit trail; X-Forwarded-For counts only when the deployment
       says a trusted proxy sets it. */
37 public static String clientIp(AppContext app, HttpServletRequest request) {
38     if (app.settings.trustForwardedFor()) {
```

**Source:** web/SecurityEvents.java, line 19. **Java features:** static utility, rate limiter reuse, never throws.

# 8. JSP and JSTL

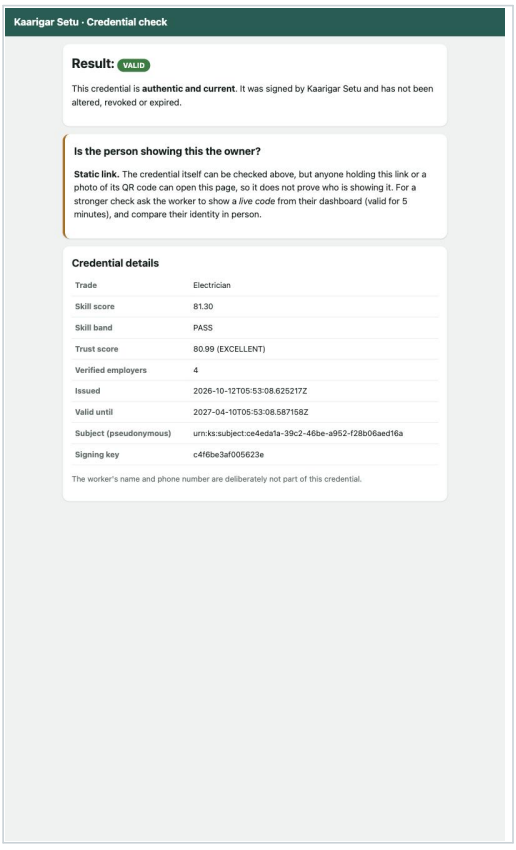
Seven JSP pages sit under WEB-INF, so they can only be reached through a page servlet. They use JSTL tags and the expression language and contain no Java scriptlets.

### Listing 28. The public verification page

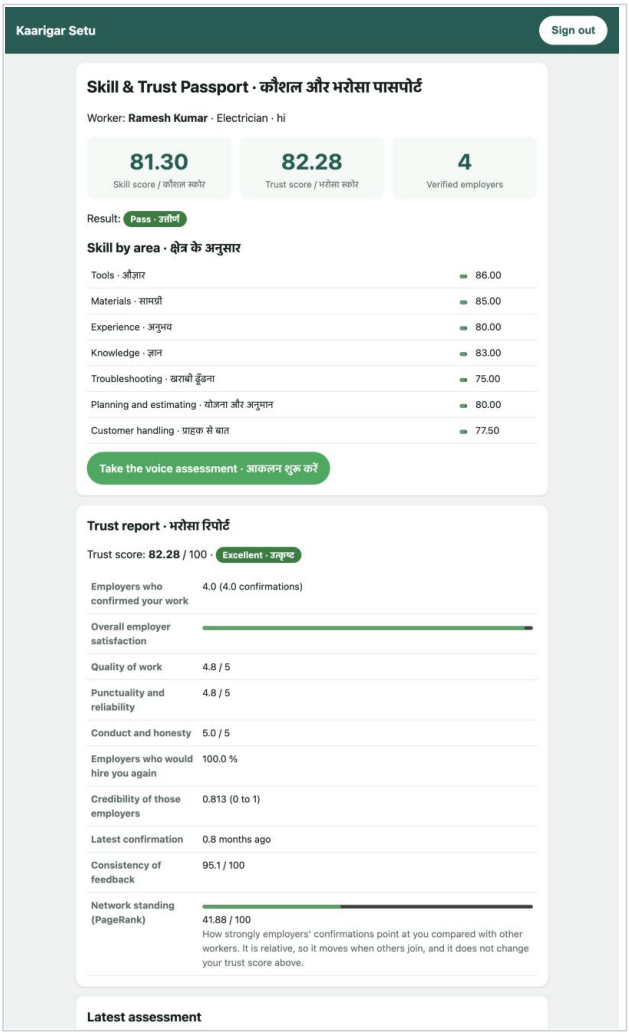
Every value is written with c:out, which HTML-escapes it, so a malicious name or value cannot inject markup. c:choose and c:when select the explanation for each result state.

```
15 <section class="card">
16   <h2>Result: <span class="badge <c:out value='${state}'/>"><c:out value="${state}"/></span></h2>
17   <c:choose>
18     <c:when test="${valid}"><p>This credential is <strong>authentic and current</strong>. It was signed by
         Kaarigar Setu and has not been altered, revoked or expired.</p></c:when>
19     <c:when test="${state == 'REVOKED'}"><p>This credential was <strong>revoked</strong> and must not be relied
         on. <c:out value="${credential.revocationReason}"/></p></c:when>
20     <c:when test="${state == 'EXPIRED'}"><p>This credential <strong>has expired</strong>. Ask the worker for a
         current one.</p></c:when>
21     <c:when test="${state == 'INVALID_SIGNATURE'}"><p><strong>The signature does not match.</strong> The
         credential may have been tampered with. Do not trust it.</p></c:when>
22     <c:otherwise><p>No credential with this reference exists.</p></c:otherwise>
23   </c:choose>
24 </section>
25 <c:if test="${state != 'NOT_FOUND'}">
26   <section class="card presentation <c:out value='${presentation}'/>">
27     <h3>Is the person showing this the owner?</h3>
28     <c:choose>
```

**Source:** web: WEB-INF/jsp/verify.jsp, line 15. **Java features:** JSP, JSTL core tags, expression language, output escaping.




*The page produced by that JSP: result, the 'is the person showing this the owner?' notice, and the pseudonymous credential details.*




*The worker passport (another JSP): skill and trust scores, trust report and PageRank network standing.*

# 9. The JavaFX moderator console

A separate Maven module. It talks to the Java backend over HTTP (java.net.http.HttpClient with a cookie jar), so the rules and the audit trail are identical to the web path. Network calls run off the UI thread.

### Listing 29. Keeping the window responsive

background() wraps a call in a JavaFX Task, runs it on a daemon thread and delivers the result back on the UI thread. A 401 returns the user to the sign-in screen instead of leaving them stuck.

```
427 private <T> void background(Supplier<T> work, Consumer<T> onSuccess, Consumer<String> onError) {
428     Task<T> task = new Task<>() {
429         @Override
430         protected T call() {
431             return work.get();
432         }
433     };
434     task.setOnSucceeded(e -> onSuccess.accept(task.getValue()));
435     task.setOnFailed(e -> {
436         Throwable failure = task.getException();
437         Platform.runLater(() -> {
438             if (failure instanceof ConsoleException console && "unauthenticated".equals(console.code()) &&
      queueShown) {
439                 // the session ended while the queue was open (timeout, server restart): go back to sign-in
      instead of staying stuck
440                 rows.clear();
441                 queueShown = false;
442                 stage.setScene(new Scene(loginPane(baseUrl, "Your session has ended. Please sign in again."),
      460, 360));
443                 return;
444             }
445             onError.accept(failure instanceof ConsoleException ? failure.getMessage() : "Unexpected error.");
446         });
447     });
448     Thread thread = new Thread(task, "console-io");
449     thread.setDaemon(true);
450     thread.start();
451 }
```

**Source:** console: ModeratorConsoleApp.java, line 427. **Java features:** javafx.concurrent.Task, Platform.runLater, generics, Supplier / Consumer, daemon Thread.

### Listing 30. Filling the Security tab

The status colour, alert labels, count table and recent-events table are all updated from one record, SecurityOverview.

```
367 private void showSecurity(SecurityOverview overview) {
368     String color = switch (overview.status()) {
369         case "OK" -> "#2e7d32";
370         case "CRITICAL" -> "#b71c1c";
371         default -> "#a15c00";
372     };
373     securityStatus.setText("Security status: " + overview.status());
374     securityStatus.setStyle("-fx-font-size: 16px; -fx-font-weight: bold; -fx-text-fill: " + color + ";");
375     alertBox.getChildren().clear();
376     if (overview.alerts().isEmpty()) {
377         Label none = new Label("No alerts: every counter is below its threshold and the audit trail verifies.");
378         none.setStyle("-fx-text-fill: #2e7d32;");
379         alertBox.getChildren().add(none);
380     }
381     for (SecurityOverview.Alert alert : overview.alerts()) {
382         Label label = new Label(alert.level() + ": " + alert.title() + " - " + alert.detail());
383         label.setWrapText(true);
384         label.setStyle("-fx-font-weight: bold; -fx-text-fill: " + (alert.level().equals("WARN") ? "#a15c00" :
      "#b71c1c") + ";");
385         alertBox.getChildren().add(label);
386     }
387     counts.setAll(overview.counts());
388     events.setAll(overview.recent());
389     integrityLine.setText(overview.integrity());
390 }
391
392 private static <T> TableColumn<T, String> plain(String title, double width, java.util.function.Function<T,
      javafx.beans.value.ObservableValue<String>> value) {
393     TableColumn<T, String> column = new TableColumn<>(title);
394     column.setPrefWidth(width);
```

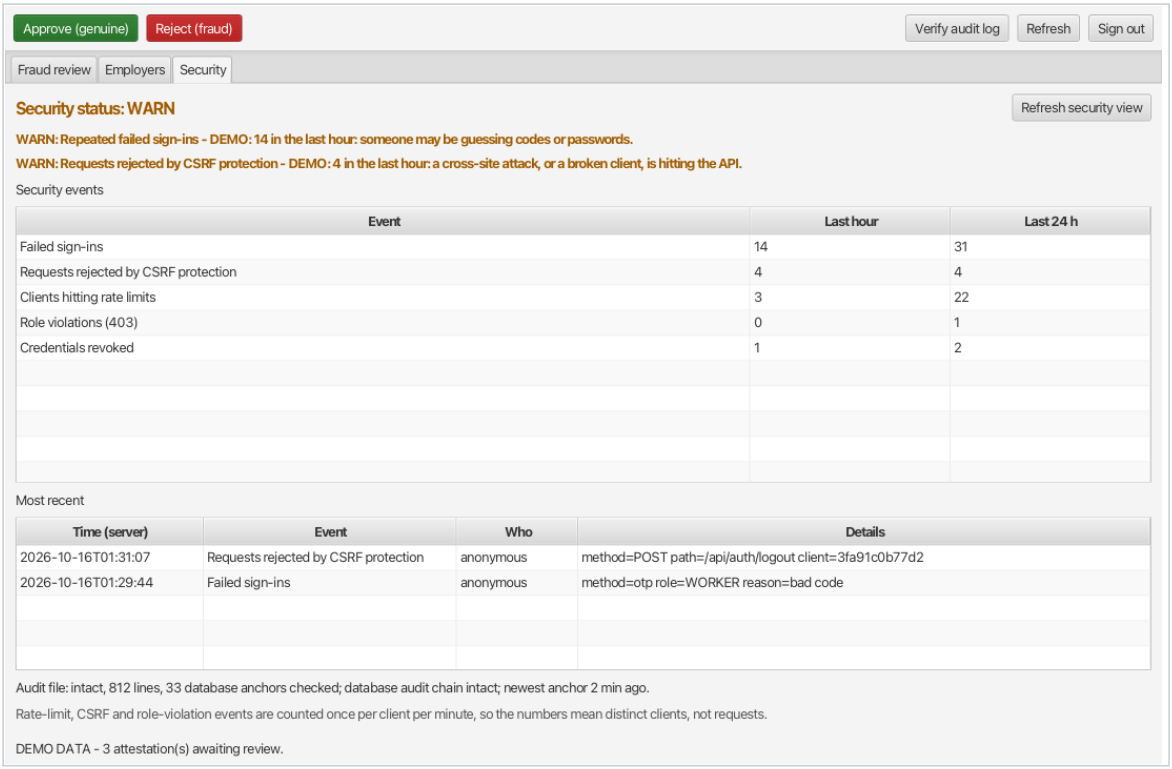
**Source:** console: ModeratorConsoleApp.java, line 367. **Java features:** TableView, ObservableList, switch expression, records, CSS styling in code.

### Listing 31. Parsing the backend's reply

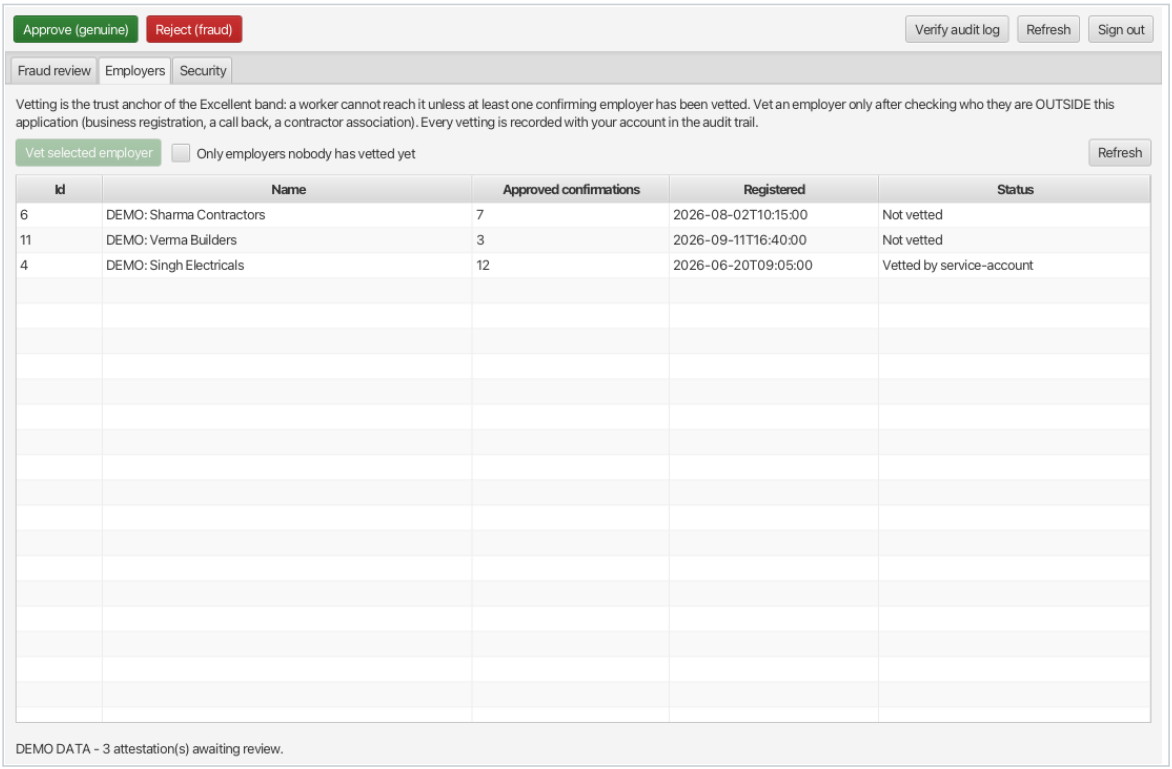
HttpModeratorApi turns the JSON into immutable records; the UI never touches HTTP or JSON directly.

```
109 public SecurityOverview securityOverview() {
110     JsonObject reply = send("GET", "api/moderator/security", null);
111     List<SecurityOverview.Alert> alerts = new ArrayList<>();
112     for (JsonElement element : reply.getAsJsonArray("alerts")) {
113         JsonObject o = element.getAsJsonObject();
114         alerts.add(new SecurityOverview.Alert(text(o, "level"), text(o, "title"), text(o, "detail")));
115     }
116     List<SecurityOverview.Count> counts = new ArrayList<>();
117     for (var entry : reply.getAsJsonObject("counts").entrySet()) {
118         JsonObject o = entry.getValue().getAsJsonObject();
119         counts.add(new SecurityOverview.Count(entry.getKey(), o.get("lastHour").getAsLong(),
      o.get("last24h").getAsLong()));
120     }
121     List<SecurityOverview.Event> recent = new ArrayList<>();
122     for (JsonElement element : reply.getAsJsonArray("recent")) {
123         JsonObject o = element.getAsJsonObject();
124         recent.add(new SecurityOverview.Event(text(o, "at"), text(o, "type"), text(o, "actor"), text(o,
      "details")));
125     }
126     JsonObject i = reply.getAsJsonObject("integrity");
127     boolean fileOk = i.get("fileChainValid").getAsBoolean();
128     String integrity = "Audit file: " + (fileOk ? "intact" : "TAMPERED (" + text(i, "fileDetail") + ")") + ", "
      + i.get("fileLines").getAsLong() + " lines, "
129           + i.get("anchorsChecked").getAsLong() + " database anchors checked"
130           + (i.has("databaseChainValid") ? "; database audit chain " +
      (i.get("databaseChainValid").getAsBoolean() ? "intact" : "BROKEN") : "")
```

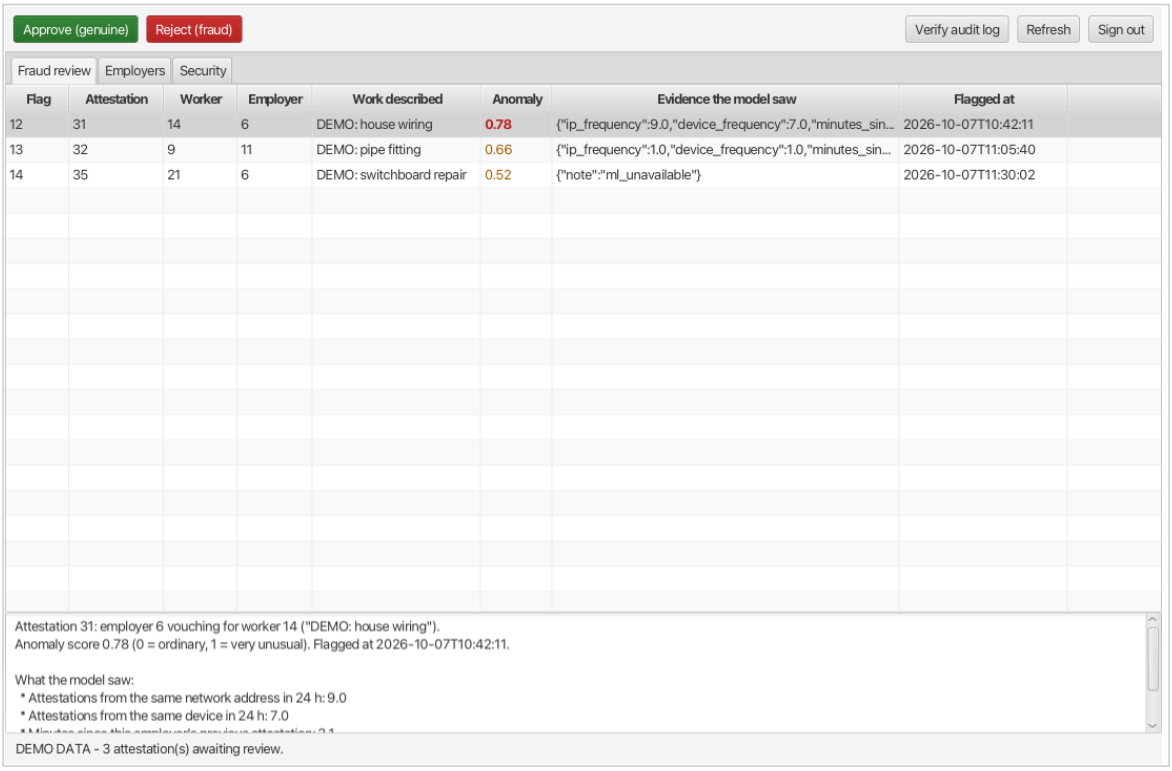
**Source:** console: HttpModeratorApi.java, line 109. **Java features:** Gson tree model, records, interface ModeratorApi for testing.




*The Security tab: status, alerts, counts, recent events and audit integrity.*




*The Employers tab: the list moderators use to vet employers.*




*The fraud review queue with the evidence the model saw.*

# 10. Cryptography and key handling with the Java platform API

Only standard Java Cryptography Architecture classes are used; no custom cryptography.

| Purpose | Java API | Parameters |
|---|---|---|
| Sign credentials | java.security.Signature, KeyStore (PKCS12) | SHA256withRSA, 3072-bit key (minimum 2048 enforced) |
| Hash passwords | SecretKeyFactory | PBKDF2WithHmacSHA256, 210,000 iterations, random salt |
| Encrypt phone numbers | javax.crypto.Cipher | AES/GCM/NoPadding, 128-bit tag, random IV, associated data |
| Look up phone numbers, sign live codes and audit heads | javax.crypto.Mac | HmacSHA256, with a distinct label in every input |
| Random values | java.security.SecureRandom | codes, salts, IVs, error reference ids |
| Compare secrets | MessageDigest.isEqual | constant time |

### Listing 32. Signing and verifying with the key ring

A credential records the id of the key that signed it. After a rotation the old public key is loaded from a directory of key files, a file whose name does not match its key's fingerprint is refused, and an unknown key id never verifies.

```
 80 public String sign(String payload) {
 81     try {
 82         Signature signer = Signature.getInstance("SHA256withRSA");
 83         signer.initSign(privateKey);
 84         signer.update(payload.getBytes(StandardCharsets.UTF_8));
 85         return Base64.getEncoder().encodeToString(signer.sign());
 86     } catch (Exception exception) {
 87         throw new IllegalStateException("Unable to sign credential", exception);
 88     }
 89 }
 90
 91 /**
 92  * Loads retired public keys from a directory of {@code <keyId>.pem} files (created by scripts/rotate-signing-
         key.sh). A file whose name does not
 93  * match the fingerprint of the key inside it is refused, so a key cannot be slipped in under another key's id.
 94  */
 95 public SignatureService withRetiredKeys(Path directory) {
 96     if (directory == null || !Files.isDirectory(directory)) return this;
 97     try (Stream<Path> files = Files.list(directory)) {
 98         for (Path file : files.filter(f -> f.getFileName().toString().endsWith(".pem")).sorted().toList()) {
 99             PublicKey key = parsePublicKeyPem(Files.readString(file, StandardCharsets.UTF_8));
100             String id = fingerprint(key);
101             String claimed =
       java.util.Objects.requireNonNull(file.getFileName()).toString().replaceFirst("\\.pem$", "");
102             if (!id.equals(claimed)) {
103                 throw new IllegalStateException("Retired key file " + file + " holds key " + id + ", not " +
       claimed);
104             }
105             if (!id.equals(keyId)) retired.put(id, key);
106         }
107     } catch (IOException | java.security.GeneralSecurityException exception) {
```

**Source:** security/SignatureService.java, line 80. **Java features:** Signature, KeyFactory, X509EncodedKeySpec, Base64, Map of retired keys.

### Listing 33. Verification by the named key

The overload that picks the key by id.

```
114 public boolean verify(String payload, String encodedSignature, String signerKeyId) {
115     PublicKey key = signerKeyId == null || signerKeyId.isBlank() || signerKeyId.equals(keyId) ? publicKey :
        retired.get(signerKeyId);
116     return key != null && verifyWith(key, payload, encodedSignature);
117 }
```

**Source:** security/SignatureService.java, line 114. **Java features:** overloaded methods, null-safe ternary.

### Listing 34. Password hashing

A random salt and 210,000 PBKDF2 rounds; the stored form carries the algorithm and iteration count so it can be upgraded later.

```
21 public static String hash(char[] password) {
22     byte[] salt = new byte[16];
23     RANDOM.nextBytes(salt);
24     return "pbkdf2-sha256$" + ITERATIONS + "$" + Base64.getEncoder().encodeToString(salt) + "$"
25             + Base64.getEncoder().encodeToString(derive(password, salt, ITERATIONS));
26 }
27
28 /** Constant-time. A malformed stored value never matches. */
29 public static boolean matches(char[] password, String stored) {
30     try {
31         String[] parts = stored.split("\\$");
32         if (parts.length != 4 || !parts[0].equals("pbkdf2-sha256")) return false;
```

**Source:** security/PasswordHasher.java, line 21. **Java features:** SecretKeyFactory, PBEKeySpec, SecureRandom, Base64.

### Listing 35. Phone-number encryption

AES-GCM with a fresh IV per value and associated data; the output is a version byte, the IV and the ciphertext.

```
71     Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
72     cipher.init(Cipher.ENCRYPT_MODE, aesKey, new GCMParameterSpec(128, iv));
73     cipher.updateAAD(AAD);
74     byte[] ciphertext = cipher.doFinal(normalizedPhone.getBytes(StandardCharsets.UTF_8));
75     byte[] out = new byte[1 + IV_BYTES + ciphertext.length];
76     out[0] = VERSION;
77     System.arraycopy(iv, 0, out, 1, IV_BYTES);
78     System.arraycopy(ciphertext, 0, out, 1 + IV_BYTES, ciphertext.length);
79     return out;
80 } catch (GeneralSecurityException exception) {
81     throw new IllegalStateException("Phone encryption failed", exception);
82 }
```

**Source:** security/PhoneProtector.java, line 71. **Java features:** Cipher, GCMParameterSpec, byte-array handling, checked-exception translation.

### Listing 36. Live codes (HMAC tokens)

A token is the issue time plus an HMAC over the credential id and that time. Check recomputes it in constant time and classifies the code as live, expired or invalid.

```
50 public Status check(UUID credentialRef, String token) {
51     if (token == null || token.isBlank()) return new Status(State.NONE, Duration.ZERO);
52     if (token.length() > MAX_TOKEN_LENGTH) return invalid();
53     int dot = token.indexOf('.');
54     if (dot < 1 || dot == token.length() - 1) return invalid();
55     long issued;
56     try {
57         issued = Long.parseLong(token.substring(0, dot));
58     } catch (NumberFormatException malformed) {
59         return invalid();
60     }
61     String expected = mac(credentialRef, issued);
62     if (!MessageDigest.isEqual(expected.getBytes(StandardCharsets.US_ASCII), token.substring(dot +
         1).getBytes(StandardCharsets.US_ASCII))) return invalid();
63     Duration age = Duration.between(Instant.ofEpochSecond(issued), clock.instant());
64     if (age.compareTo(CLOCK_SKEW.negated()) < 0) return invalid();                    // minted "in the
         future": a clock problem, never trust it
65     if (age.compareTo(LIFETIME) > 0) return new Status(State.EXPIRED, age);
66     return new Status(State.LIVE, age.isNegative() ? Duration.ZERO : age);
67 }
68
69 public Instant expiresAt(String token) {
```

**Source:** security/PresentationTokens.java, line 50. **Java features:** Mac, enum, record Status, java.time.Duration, Clock injection.

### Listing 37. Canonical numbers

Scores such as 81.30 and 81.3 become the same text so a signature survives re-serialisation by other JSON libraries; anything outside the supported profile is refused rather than guessed.

```
70 static String number(BigDecimal value) {
71     if (value.signum() == 0) return "0";
72     BigDecimal plain = value.stripTrailingZeros();
73     BigDecimal magnitude = plain.abs();
74     if (plain.precision() > 15 || magnitude.compareTo(UPPER) >= 0 || magnitude.compareTo(LOWER) < 0) {
75         throw new IllegalArgumentException("number outside the canonical profile: " + value);
76     }
77     return plain.toPlainString();
78 }
```

**Source:** util/CanonicalJson.java, line 70. **Java features:** BigDecimal.stripTrailingZeros, toPlainString, IllegalArgumentException.

# 11. Concurrency and scheduled work

The application serves many requests at once, so shared state is protected explicitly.

| Mechanism | Where | Why |
|---|---|---|
| synchronized methods | AttestationStore.add, InMemoryOtpChallengeStore, AuditLogger.anchorFileHead, AuditLogWriter.append | check-then-act sequences must not interleave |
| ReentrantLock + FileLock | ChainedAuditFileSink | serialise writers inside the process and across processes |
| ConcurrentHashMap.compute | RateLimiter | atomic per-key window update without a global lock |
| ThreadLocal | PooledConnectionProvider | bind a transaction to the current request thread |
| ScheduledExecutorService | AppContext | background jobs on a daemon thread |

### Listing 38. Background timers

Two scheduled jobs: clearing old rate-limit counters, and pinning the audit file's newest line in the database every five minutes. A failure in either is logged and never stops the timer.

```
140     scheduler.scheduleWithFixedDelay(() -> limiter.evictExpired(Duration.ofHours(2)), 10, 30, TimeUnit.MINUTES);
141     // Pin the audit file's newest line in the database every few minutes (and once soon after start), so a
          rolled-back file is caught.
142     scheduler.scheduleWithFixedDelay(() -> {
143         try {
144             audit.anchorFileHead();
145         } catch (RuntimeException failure) {
146             System.getLogger("ks.audit").log(System.Logger.Level.WARNING, "audit anchoring failed: " +
          failure.getMessage());
147         }
148     }, 20, 300, TimeUnit.SECONDS);
149 }
```

**Source:** web/AppContext.java, line 140. **Java features:** ScheduledExecutorService, TimeUnit, lambdas, System.Logger.

### Listing 39. AuditLogger.anchorFileHead

Writes only to the database, never to the audit file, so anchoring cannot feed back into itself; synchronized so two threads cannot anchor the same line twice.

```
67 public synchronized boolean anchorFileHead() {
68     try {
69         java.util.Optional<AuditFileSink.Head> head = fileSink.head();
70         if (head.isEmpty() || head.get().seq() <= lastAnchoredSeq) return false;
71         repository.save(new AuditLog(null, ANCHOR_EVENT, "system", "seq=" + head.get().seq() + " hash=" +
        head.get().lineHash(), LocalDateTime.now(clock)));
72         lastAnchoredSeq = head.get().seq();
73         sinceAnchor = 0;
74         return true;
75     } catch (IOException exception) {
76         throw new IllegalStateException("Unable to anchor the audit file", exception);
77     }
78 }
79
80 /** Anchors recorded in the database, newest first. Rows that do not parse are ignored (they cannot be forged
        into acceptance, only missed). */
81 public java.util.List<ChainedAuditFileSink.Anchor> anchors(int limit) {
```

**Source:** security/AuditLogger.java, line 67. **Java features:** synchronized, Optional, records, exception translation.

# 12. Java built for the security review

Each item below began as a weakness found by attacking the running application, and each was fixed in Java with a test that would fail without it.

| Weakness found | Java written | Class |
|---|---|---|
| Credentials could not be checked by other tools (a score printed as 81.30) | RFC 8785 canonical JSON, key ring, rotation, signed revocation list | CanonicalJson, SignatureService, CredentialService |
| An old audit file could be restored undetected | anchors pinned in the database; offline verifier | ChainedAuditFileSink, AuditLogger, AuditVerifyCli |
| A screenshot of the QR works forever | HMAC live codes bound to one credential | PresentationTokens, CredentialServlet |
| Attacks were invisible | audited security events, alert rules, console tab | SecurityEvents, SecurityMonitor, ModeratorServlet |
| Three fake employer accounts reached Excellent | moderator vetting; top band needs a vetted employer | EmployerDao.vet, ModeratorService, ReputationScoring |
| Sign-up flooding and public flooding | per-address throttles | BaseServlet.throttle, WorkerServlet, CredentialServlet |

### Listing 40. Alert rules as a pure function

The thresholds are evaluated by a static method with no I/O, so every rule is unit-tested by feeding it counts.

```
56 static List<Alert> alerts(Map<String, Long> lastHour, boolean fileIntact, String fileDetail, Optional<Boolean>
       databaseIntact, Optional<Duration> anchorAge,
57                           boolean fileHasUnanchoredLines) {
58     List<Alert> alerts = new ArrayList<>();
59     if (!fileIntact) alerts.add(new Alert("CRITICAL", "Audit file integrity check FAILED", fileDetail + ". Treat
          the audit trail as compromised and preserve the files."));
60     if (databaseIntact.isPresent() && !databaseIntact.get()) alerts.add(new Alert("CRITICAL", "Database audit
         chain FAILED", "the append-only audit table no longer verifies."));
61     for (Rule rule : RULES) {
62         long n = lastHour.getOrDefault(rule.eventType(), 0L);
63         if (n >= rule.highAt()) alerts.add(new Alert("HIGH", rule.title(), n + " in the last hour: " +
         rule.meaning() + "."));
64         else if (n >= rule.warnAt()) alerts.add(new Alert("WARN", rule.title(), n + " in the last hour: " +
         rule.meaning() + "."));
65     }
66     if (anchorAge.isPresent() && anchorAge.get().compareTo(ANCHOR_STALE_AFTER) > 0 && fileHasUnanchoredLines) {
67         alerts.add(new Alert("WARN", "Audit anchoring has stalled", "the newest anchor in the database is " +
         anchorAge.get().toMinutes() + " minutes old, so a file rollback would go unnoticed for longer."));
68     }
69     return alerts;
70 }
71
72 static String status(List<Alert> alerts) {
73     if (alerts.stream().anyMatch(a -> a.level().equals("CRITICAL"))) return "CRITICAL";
74     return alerts.isEmpty() ? "OK" : "WARN";
75 }
```

**Source:** moderator/SecurityMonitor.java, line 56. **Java features:** static pure function, records, streams, List.of rules.

### Listing 41. The vetting rule in the scoring code

If no confirming employer was vetted by a moderator, the score is held just under Excellent. The rule is one parameter, so it can be switched off in a test.

```
172 boolean anyVetted = employers.keySet().stream().anyMatch(id -> standing.getOrDefault(id,
        EmployerStanding.unknown()).verified());
173 boolean heldBack = p.excellentNeedsVettedEmployer() && !anyVetted && score.doubleValue() > HELD_BELOW_EXCELLENT;
174 if (heldBack) score = BigDecimal.valueOf(HELD_BELOW_EXCELLENT).setScale(2, RoundingMode.HALF_UP);
```

**Source:** trust/ReputationScoring.java, line 172. **Java features:** streams, parameter record, BigDecimal.

### Listing 42. An attack simulation that calls the real scoring code

The simulation builds fake employers with chosen capabilities and asks how many it takes to reach each band, before and after the vetting rule. It proved that three throwaway accounts used to be enough.

```
public static ReputationScoring.Report attack(Capability capability, int employers, int perEmployer, int
      fraudRulings, ReputationScoring.Params params) {
    List<ReputationScoring.Feedback> feedback = new ArrayList<>();
    Map<Long, ReputationScoring.EmployerStanding> standing = new HashMap<>();
    long id = 1;
    for (long e = 1; e <= employers; e++) {
        ReputationScoring.EmployerStanding s = capability.standing;
        standing.put(e, new ReputationScoring.EmployerStanding(s.verified(), s.approvedGiven(), fraudRulings > 0
       && e == 1 ? 1 : 0, s.networkStanding()));
        for (int n = 0; n < perEmployer; n++) {
            feedback.add(new ReputationScoring.Feedback(id++, 1L, e, 5, 5, 5, true, 30, NOW.minusSeconds(10L *
      86400 + n * 3600L), 0.0));
        }
    }
    return ReputationScoring.score(feedback, standing, ReputationScoring.calibrate(feedback, params),
      fraudRulings, NOW, params);
}

/** Smallest number of fake employers (one confirmation each) that reaches the score, or -1 if even {@code
      limit} employers do not. */
public static int employersNeeded(Capability capability, double target, int limit) {
    return employersNeeded(capability, target, limit, ReputationScoring.Params.defaults());
}
```

**Source:** test: trust/TrustAttackSimulation.java, line 50. **Java features:** enum with fields, records, parameter record, loops over production code.

# 13. Testing the Java code

322 JUnit 5 tests in the web application and 13 in the console, using Mockito for stand-ins and a real PostgreSQL database for the integration tests (the test database is wiped, so its name must end in _test).

| Kind of test | Examples | What it protects |
|---|---|---|
| Unit tests with stand-ins | ReputationScoringTest (31), CredentialServiceTest, AuthServiceTest | business rules and security rules in isolation |
| Database integration | PipelineDaoIntegrationTest, DaoIntegrationTest, ErasureIntegrationTest | SQL, triggers, constraints, the audit chain, the restricted role |
| Fuzz by reflection | DaoSqlInjectionFuzzTest | no DAO method can be injected |
| Architecture guards | NoStringBuiltSqlTest, StaticAssetsTest | a rule about the code base itself fails the build |
| Security design tests | SignatureKeyRotationTest, CanonicalJsonTest, PresentationTokensTest, TrustAttackSimulationTest, SecurityMonitorTest | the controls added for the security review |

### Listing 43. SQL-injection fuzz by reflection

The test discovers every public method of every DAO that takes a String, calls it with injection payloads in each string position, and then checks that no table appeared or disappeared and no existing row changed.

```
137 for (Object dao : daos) {
138     for (Method method : dao.getClass().getDeclaredMethods()) {
139         if (!Modifier.isPublic(method.getModifiers()) || method.isSynthetic()) continue;
140         Class<?>[] types = method.getParameterTypes();
141         boolean hasString = false;
142         boolean allSupported = true;
143         for (Class<?> type : types) {
144             hasString |= type == String.class;
145             allSupported &= supported(type);
146         }
147         String name = dao.getClass().getSimpleName() + "." + method.getName();
148         if (!hasString) continue;
149         if (!allSupported) { skipped.add(name); continue; }
150         methodsWithStrings++;
151         for (int position = 0; position < types.length; position++) {
152             if (types[position] != String.class) continue;
153             for (String payload : PAYLOADS) {
154                 Object[] args = new Object[types.length];
155                 for (int i = 0; i < types.length; i++) args[i] = i == position ? payload : harmless(types[i]);
156                 calls++;
157                 try {
158                     method.invoke(dao, args);
```

**Source:** test: dao/DaoSqlInjectionFuzzTest.java, line 137. **Java features:** reflection (Method.invoke, getDeclaredMethods), generics, collections, JDBC.

### Listing 44. A test about the source code

This test reads every Java source file and fails the build if any SQL string is built by concatenation or any Statement is created, so the prepared-statement rule cannot silently erode.

```
@Test
void noStatementObjectsAreEverUsed() throws IOException {
    for (Path file : sources()) {
        String text = Files.readString(file);
        assertFalse(text.contains(".createStatement("), file + " uses java.sql.Statement; use
      PreparedStatement");
        assertFalse(Pattern.compile("\\.execute(Query|Update)?\\(\\s*\"").matcher(text).find() &&
      text.contains("java.sql.Statement ") ,
                file + " executes a raw SQL string");
    }
}

@Test
void everySqlAssignmentAndPrepareCallIsBuiltFromLiteralsOnly() throws IOException {
    List<String> offenders = new ArrayList<>();
    int checked = 0;
    for (Path file : sources()) {
        String text = Files.readString(file);
        Matcher assignment = SQL_ASSIGNMENT.matcher(text);
        while (assignment.find()) {
            checked++;
            if (!residue(assignment.group(1)).isEmpty()) offenders.add(file + ": String sql = " +
      assignment.group(1).strip());
        }
        Matcher prepare = PREPARE.matcher(text);
        while (prepare.find()) {
            String argument = firstArgument(text, prepare.end()).strip();
```

**Source:** test: architecture/NoStringBuiltSqlTest.java, line 62. **Java features:** Files.readString, regular expressions, JUnit assertions with messages.

### Listing 45. Key rotation

Signs with one key, retires it, loads a new key ring and checks the old credential still verifies and a forged key id does not.

```
void aCredentialSignedBeforeARotationStillVerifiesAfterwardsAndNewOnesUseTheNewKey(@TempDir Path dir) throws
      Exception {
    SignatureService oldKey = new SignatureService(rsa(2048));
    String signedByOld = oldKey.sign("{\"id\":\"old\"}");
    retire(oldKey, dir);

    SignatureService newKey = new SignatureService(rsa(2048)).withRetiredKeys(dir);
    assertTrue(newKey.verify("{\"id\":\"old\"}", signedByOld, oldKey.keyId()), "old credential must survive the
      rotation");
    assertFalse(newKey.verify("{\"id\":\"old\"}", signedByOld), "the plain check still means: signed by the
      ACTIVE key");
    String signedByNew = newKey.sign("{\"id\":\"new\"}");
    assertTrue(newKey.verify("{\"id\":\"new\"}", signedByNew, newKey.keyId()));
    assertEquals(java.util.List.of("active", "retired"),
      newKey.keys().stream().map(SignatureService.KeyInfo::status).toList());
}

@Test
```

**Source:** test: credential/SignatureKeyRotationTest.java, line 31. **Java features:** @TempDir, KeyPairGenerator, JUnit 5.

### Listing 46. A test taken from the RFC

Checks the member ordering example from RFC 8785 (surrogate pairs sort before a later BMP character).

```
void membersAreOrderedByUtf16CodeUnitsAsInTheRfcExample() {
    // RFC 8785 section 3.2.3: carriage return, "1", U+0080, o-diaeresis, euro, emoji (a surrogate pair, D83D),
      Hebrew dalet with dagesh (FB33)
    String json = "{\"\\u20ac\":\"Euro\",\"\\r\":\"CR\",\"\\ufb33\":\"Dalet\",\"1\":\"One\",\"\\ud83d\\ude00\":\
      "Emoji\",\"\\u0080\":\"Ctl\",\"\\u00f6\":\"o\"}";
    String out = canon(json);
    int[] positions = {out.indexOf("\"\\r\""), out.indexOf("\"1\""), out.indexOf("\"\u0080\""),
      out.indexOf("\"\u00f6\""), out.indexOf("\"\u20ac\""),
          out.indexOf("\"\ud83d\ude00\""), out.indexOf("\"\ufb33\"")};
    for (int i = 1; i < positions.length; i++) assertTrue(positions[i - 1] >= 0 && positions[i - 1] <
      positions[i], "member " + i + " out of order in " + out);
}
```

**Source:** test: util/CanonicalJsonTest.java, line 46. **Java features:** unicode escapes, ordering by UTF-16 code units.

### Listing 47. Every page's script and style must exist

Added after a missing script silently broke audio recording: the build now fails if a JSP refers to a file that is not there.

```
28 void everyStaticFileReferencedByAPageExists() throws IOException {
29     List<String> missing = new ArrayList<>();
30     int references = 0;
31     try (Stream<Path> pages = Files.walk(WEBAPP.resolve("WEB-INF/jsp"))) {
32         for (Path page : pages.filter(p -> p.toString().endsWith(".jsp")).toList()) {
33             Matcher matcher = REFERENCE.matcher(Files.readString(page, StandardCharsets.UTF_8));
34             while (matcher.find()) {
35                 references++;
36                 if (!Files.isRegularFile(WEBAPP.resolve(matcher.group().substring(1))))
       missing.add(page.getFileName() + " -> " + matcher.group());
37             }
38         }
39     }
40     assertTrue(references >= 10, "the scan should have found the pages' scripts and styles, found " +
       references);
41     assertEquals(List.of(), missing, "pages reference files that do not exist");
42 }
43
44 @Test
```

**Source:** test: web/StaticAssetsTest.java, line 28. **Java features:** Files.walk, regex, streams, assertEquals on a list.

```
$ cd kaarigarsetu-java && mvn test
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  in credential.QRGeneratorTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0  in credential.SignatureKeyRotationTest
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0  in credential.CredentialServiceTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  in util.WavValidatorTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0  in util.CanonicalJsonTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  in util.FileUtilitiesTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0  in security.PresentationTokensTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0  in security.ChainedAuditFileSinkTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  in security.PasswordHasherTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0  in security.RateLimiterTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0  in security.SignatureServiceKeystoreTest
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0  in security.OtpServiceTest
Tests run: 12, Failures: 0, Errors: 0, Skipped: 0  in security.AuthServiceTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0  in security.PhoneProtectorTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0  in security.OtpAndSignatureServiceTest
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0  in security.AuditLoggerTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0  in security.HttpSmsOtpSenderTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  in web.SessionsTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0  in web.AuthFilterTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  in web.CsrfFilterTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  in web.StaticAssetsTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  in moderator.SecurityMonitorTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0  in moderator.ModeratorServiceTest
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0  in trust.TrustScoreServiceTest
Tests run: 31, Failures: 0, Errors: 0, Skipped: 0  in trust.ReputationScoringTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  in trust.TrustGraphServiceTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  in trust.TrustAttackSimulationTest
Tests run: 322, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

$ cd kaarigarsetu-moderator-console && mvn test
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

$ cd kaarigarsetu-ml-service && python -m pytest -q
370 passed in 2.77s
```

*Test results: 322 Java tests, 13 console tests, 370 Python tests.*

```
$ mvn -Psast compile spotbugs:check      # SpotBugs + find-sec-bugs
exit code: 0 (0 = no findings; 73 pattern-noise items excluded with reasons in spotbugs-exclude.xml)

$ bandit -q -r kaarigarsetu-ml-service/app -ll
exit code: 0

$ semgrep scan --config p/java --config p/python --config p/javascript --config p/owasp-top-ten --config p/secrets --error ...
exit code: 0 (0 = no findings in 213 files)

$ gitleaks dir <tree that ships in the zip> -c .gitleaks.toml
11:15PM INF no leaks found

$ pip-audit -r requirements.txt  /  -r requirements-faster-whisper.txt
No known vulnerabilities found
No known vulnerabilities found

$ python3 scripts/osv_scan_maven.py
checked 15 dependencies, 0 problem(s)
```

*Static analysis and dependency scans of the Java code (SpotBugs with find-sec-bugs and OSV among them).*

# 14. Where the code differs from the Review 1 slides

The slides were sketches. The finished code is more developed and in a few places named differently.

| Slide | The finished code |
|---|---|
| abstract int evaluateResponse(String) | The abstract members are supportedTrade() and feedbackFor(score). The shared algorithms assess() and summarize() are final. Scoring each spoken answer is done by the ML service; the subclass averages and words the result. |
| InvalidOTPException extends Exception (checked) | All user-defined exceptions extend RuntimeException and are translated in one place (ApiErrors). |
| Map and a TreeMap leaderboard | Not implemented. Trust is computed per worker by ReputationScoring and stored in the database; the adjacency list (TrustGraphService) is implemented. |
| Files.writeString(Paths.get("audit.log")) | AuditLogWriter is that simple writer; the audit trail actually used is the hash-chained ChainedAuditFileSink plus an append-only database table. |
| Servlet at /attestation with request parameters | Servlets live under /api/..., read JSON bodies and require a CSRF token and a role. |
| JSP with scriptlets (<%= ... %>) | JSP uses JSTL and the expression language with output escaping; no scriptlets. |
| JavaFX TableView with approve and reject | Implemented and extended: sign-in with a one-time code, an Employers (vetting) tab and a Security tab. |
| Electrician and Plumber | Six trades: electrician, plumber, carpenter, mason, welder, painter. The rubrics are placeholders marked TEST ONLY until an expert supplies real ones. |

# 15. Index of listings by Java feature

| No. | Listing | Source file and line | Java features |
|---|---|---|---|
| 1 | Filter order is declared in web.xml | web: WEB-INF/web.xml:9 | Servlet Filter API, declarative configuration |
| 2 | AssessmentService: one shared rule, two things each trade must supply | assessment/AssessmentService.java:12 | abstract class, final methods, template method, BigDecimal arithmetic, defensive validation |
| 3 | A trade subclass: small and only about wording | assessment/PlumberAssessmentService.java:5 | inheritance, @Override, protected hook method |
| 4 | Polymorphic dispatch through a common reference | assessment/AssessmentScoringDispatcher.java:15 | polymorphism, streams, Optional.orElseThrow, constructor-injected List |
| 5 | InvalidOTPException | exception/InvalidOTPException.java:3 | user-defined exception, unchecked exception |
| 6 | Throwing the exception from a switch expression | security/OtpService.java:95 | switch expression with arrows, enum, throw expression |
| 7 | DuplicateAttestationException and a synchronised check | attestation/AttestationStore.java:23 | synchronized, streams (anyMatch), user-defined exception |
| 8 | ApiErrors: every exception becomes a status and a short code | web/ApiErrors.java:35 | pattern matching for instanceof (Java 16), System.Logger, HexFormat, SecureRandom |
| 9 | The trust graph as an adjacency list | trust/TrustGraphService.java:12 | Map>, HashMap, computeIfAbsent, List.copyOf / Map.copyOf, record Edge |
| 10 | Sliding-window rate limiter | security/RateLimiter.java:21 | ConcurrentHashMap.compute, ArrayDeque, lambdas, custom exception carrying retry-after |
| 11 | Canonical JSON: a TreeMap sorts members | util/CanonicalJson.java:54 | TreeMap, Map.Entry, recursion over JsonElement |
| 12 | Records for immutable data | trust/ReputationScoring.java:36 | records with static factory, compact immutable carriers, boxed Integer / Boolean for 'absent' |
| 13 | A text block for SQL | dao/EmployerDao.java:133 | text blocks, try-with-resources, PreparedStatement, record result type |
| 14 | The simple append-only audit writer | util/AuditLogWriter.java:29 | java.nio.file.Files, StandardOpenOption, Clock, synchronized |
| 15 | The hash-chained audit file | security/ChainedAuditFileSink.java:52 | FileChannel, FileLock, StandardOpenOption.APPEND, ReentrantLock, try-with-resources, JSON via Gson |
| 16 | Anchors: catching a rollback | security/ChainedAuditFileSink.java:131 | MessageDigest.isEqual (constant-time), streams, Comparator, records |
| 17 | Exporting the signed credential | util/CredentialExporter.java:11 | Files.writeString, Path, UTF-8 charsets |
| 18 | An offline auditor tool | admin/AuditVerifyCli.java:50 | main method, exit codes, JDBC from a command-line tool |
| 19 | JdbcDaoSupport: what every DAO shares | dao/JdbcDaoSupport.java:12 | abstract class, Optional, java.sql types |
| 20 | A DAO method with an array parameter | dao/AuditLogDao.java:64 | PreparedStatement, createArrayOf, try-with-resources, LinkedHashMap |
| 21 | Thread-bound transactions on a connection pool | config/PooledConnectionProvider.java:43 | HikariCP, ThreadLocal, generics with Supplier, commit / rollback, addSuppressed, finally |
| 22 | BaseServlet.service: the single error boundary | web/BaseServlet.java:26 | abstract class extends HttpServlet, template method, exception translation |

| No. | Listing | Source file and line | Java features |
|---|---|---|---|
| 23 | A one-line throttle for public routes | web/BaseServlet.java:110 | protected helper, java.time.Duration, hashed client address |
| 24 | AttestationServlet.doPost | attestation/AttestationServlet.java:22 | @WebServlet, switch with arrows, java.time.LocalDate, custom exceptions |
| 25 | AuthFilter: deny by default | security/AuthFilter.java:81 | jakarta.servlet.Filter, records, streams, Optional, Set |
| 26 | CsrfFilter | web/CsrfFilter.java:25 | Filter, constant-time token comparison, URI parsing |
| 27 | SecurityEvents: audit without a flood | web/SecurityEvents.java:19 | static utility, rate limiter reuse, never throws |
| 28 | The public verification page | web: WEB-INF/jsp/verify.jsp:15 | JSP, JSTL core tags, expression language, output escaping |
| 29 | Keeping the window responsive | console: ModeratorConsoleApp.java:427 | javafx.concurrent.Task, Platform.runLater, generics, Supplier / Consumer, daemon Thread |
| 30 | Filling the Security tab | console: ModeratorConsoleApp.java:367 | TableView, ObservableList, switch expression, records, CSS styling in code |
| 31 | Parsing the backend's reply | console: HttpModeratorApi.java:109 | Gson tree model, records, interface ModeratorApi for testing |
| 32 | Signing and verifying with the key ring | security/SignatureService.java:80 | Signature, KeyFactory, X509EncodedKeySpec, Base64, Map of retired keys |
| 33 | Verification by the named key | security/SignatureService.java:114 | overloaded methods, null-safe ternary |
| 34 | Password hashing | security/PasswordHasher.java:21 | SecretKeyFactory, PBEKeySpec, SecureRandom, Base64 |
| 35 | Phone-number encryption | security/PhoneProtector.java:71 | Cipher, GCMParameterSpec, byte-array handling, checked-exception translation |
| 36 | Live codes (HMAC tokens) | security/PresentationTokens.java:50 | Mac, enum, record Status, java.time.Duration, Clock injection |
| 37 | Canonical numbers | util/CanonicalJson.java:70 | BigDecimal.stripTrailingZeros, toPlainString, IllegalArgumentException |
| 38 | Background timers | web/AppContext.java:140 | ScheduledExecutorService, TimeUnit, lambdas, System.Logger |
| 39 | AuditLogger.anchorFileHead | security/AuditLogger.java:67 | synchronized, Optional, records, exception translation |
| 40 | Alert rules as a pure function | moderator/SecurityMonitor.java:56 | static pure function, records, streams, List.of rules |
| 41 | The vetting rule in the scoring code | trust/ReputationScoring.java:172 | streams, parameter record, BigDecimal |
| 42 | An attack simulation that calls the real scoring code | test: trust/TrustAttackSimulation.java:50 | enum with fields, records, parameter record, loops over production code |
| 43 | SQL-injection fuzz by reflection | test: dao/DaoSqlInjectionFuzzTest.java:137 | reflection (Method.invoke, getDeclaredMethods), generics, collections, JDBC |
| 44 | A test about the source code | test: architecture/NoStringBuiltSqlTest.java:62 | Files.readString, regular expressions, JUnit assertions with messages |
| 45 | Key rotation | test: credential/SignatureKeyRotationTest.java:31 | @TempDir, KeyPairGenerator, JUnit 5 |
| 46 | A test taken from the RFC | test: util/CanonicalJsonTest.java:46 | unicode escapes, ordering by UTF-16 code units |
| 47 | Every page's script and style must exist | test: web/StaticAssetsTest.java:28 | Files.walk, regex, streams, assertEquals on a list |